Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

1 Introduction to Reconnaissance

- reconnaissance describes the process of gathering information about a target before attacking
- incredible crucial step
  - "*Know the enemy and know yourself, and you can fight a hundred battles without disaster.*" - Art of War
- in order to effectively attack, we first need to get an idea what we are dealing with
  - identify attack vectors

1 Introduction to Reconnaissance - Active vs. Passive

- passive reconnaissance
  - collecting information without interacting directly with the target
- active reconnaissance
  - directly interacting with the target, which could potentially alert the target of the probing
  - his includes techniques like pinging a server or sending crafted packets to gather data.

- active reconnaissance can be divided into two sub-steps
  - scanning: which systems exist and what services are they running
  - enumeration: collecting information from open services

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

2 Active Reconnaissance - Scanning Overview

- generally, we can divide scanning into one three categories
  - host scans
  - service/port scans
  - vulnerability scans

2 Active Reconnaissance - Nmap

- Nmap stands for Network Mapper
- amazing tool with great documentation
- the go-tool utility for scanning and enumeration
- command line utility: `nmap`

2 Active Reconnaissance - Host Discovery

- we start by introducing, how we can specify target with Nmap
  - we will use IP addresses, but Nmap can also deal with hostnames
  - if it is given hostname, it will perform DNS lookup to get IP
  - given an IP that is online from the PoV of Nmap, it will also try to perform a reverse DNS lookup to get the hostname
- default behavior of Nmap
  - first perform host discovery, then port scans

2 Nmap - General

```
# target specification syntax
[user@kali ~]$ nmap <ip>
[user@kali ~]$ nmap <ip1> <ip2> …
[user@kali ~]$ nmap <ip>/<numbits>
[user@kali ~]$ nmap 102.168.0-255.0-31
[user@kali ~]$ nmap -iL <file>
[user@kali ~]$ nmap <ip>/<numbits> --exclude <host1>,<host2>
[user@kali ~]$ nmap <ip>/<numbits> --excludefile <file>

# no DNS resolution
[user@kali ~]$ nmap <ip> -n

# dns resolution against all targets, even offline
[user@kali ~]$ nmap <ip> -R
```

2 Nmap - General

```
# display all targets that would be scanned, without
# actually scanning them
[user@kali ~]$ nmap <ip>/<numbits> -sL

# do not perform host discovery
[user@kali ~]$ nmap <ip>/<numbits> -Pn

# do not perform port scan
[user@kali ~]$ nmap <ip>/<numbits> -sn
```

2 Active Reconnaissance - Host Discovery

- a host-discovery scan has the goal of identifying all online systems in some network
- to discover whether there exists an alive system at an IP address, multiple ways
  - ideas?

2 Active Reconnaissance - Host Discovery Techniques

Switch	Method
`-PR`	using the ARP protocol
`-PE`	ICMP echo request ping
`-PP`	ICMP timestamp request ping
`-PM`	ICMP mask request ping
`-PS [port1,port2,...]`	TCP SYN ping (default ports are 80 & 443)
`-PA [port1,port2,...]`	TCP ACK ping (default port is 80)
`-PU [port1,port2,...]`	UDP ping (will try multiple uncommon ones)

- any idea why TCP based scan uses common ports and UDP uses uncommon ports?

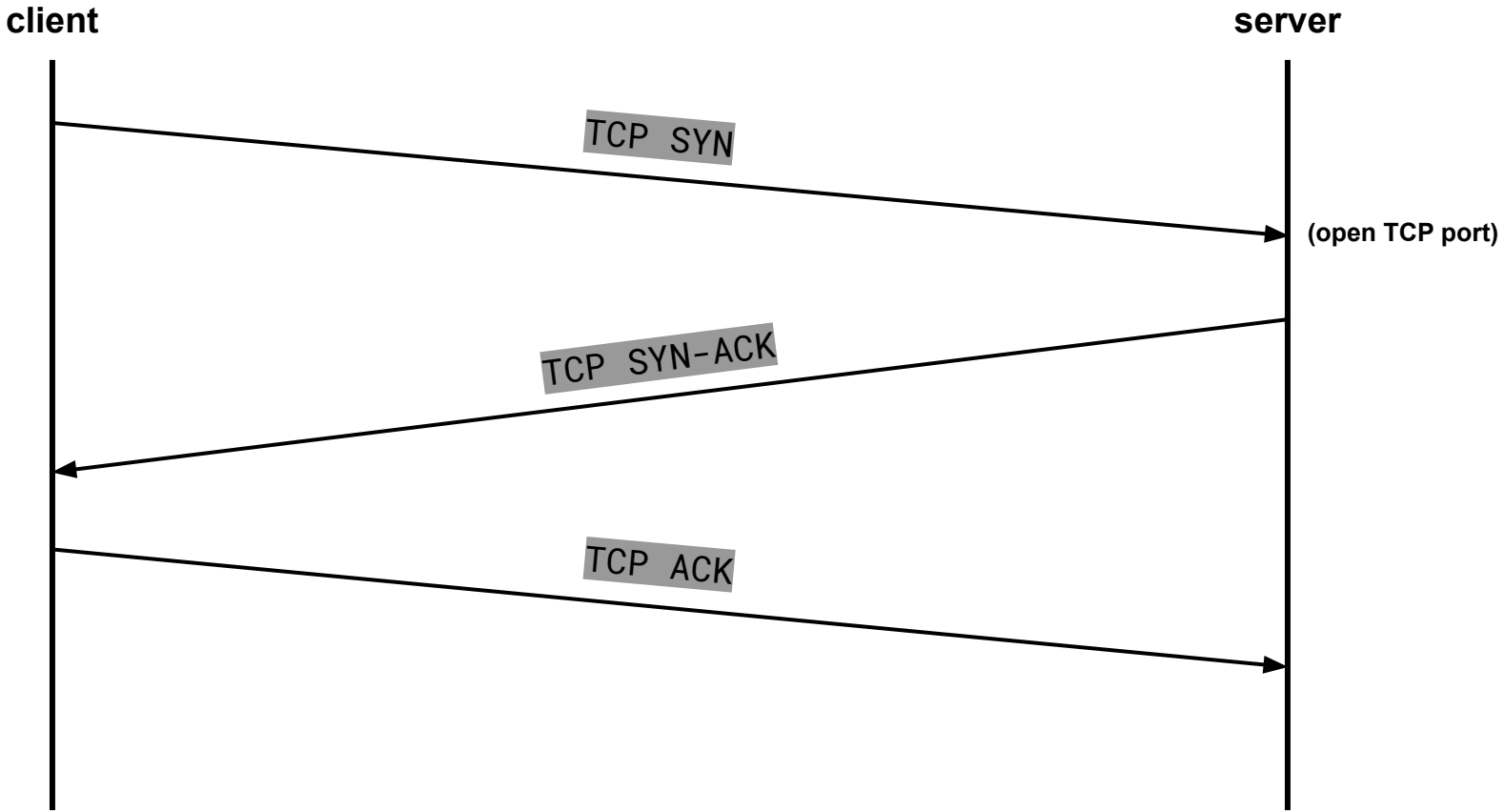
2 Active Reconnaissance - Host Discovery Default Behavior

Method	Discovery Technique
privileged user; target in local network	ARP
privileged user, target not in local network	ICMP Echo, ICMP Timestamp, TCP ACK
unprivileged user	TCP Connect

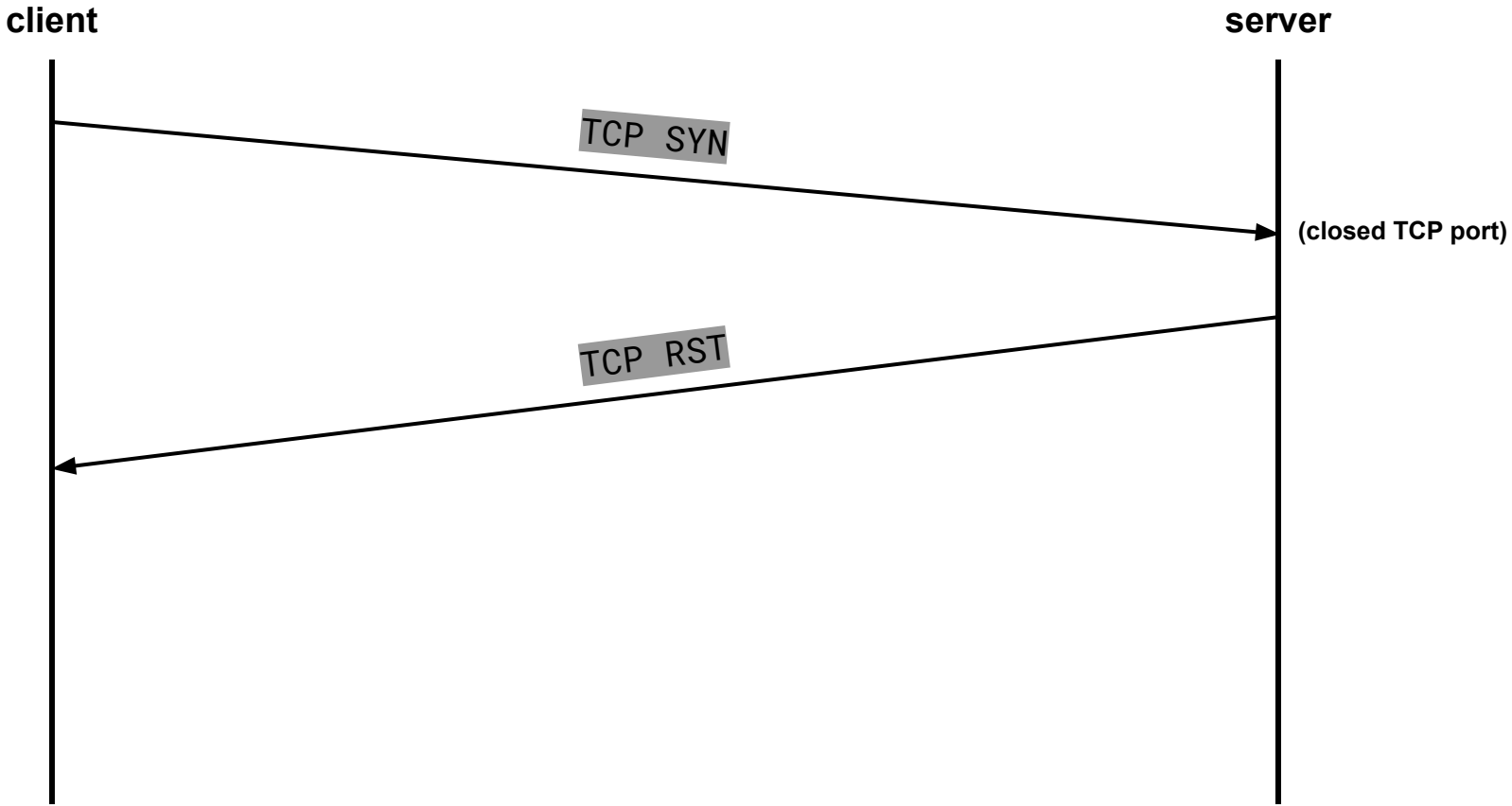
2 Active Reconnaissance - Port Scan

- a port scan has the goal of identifying all open ports on a system

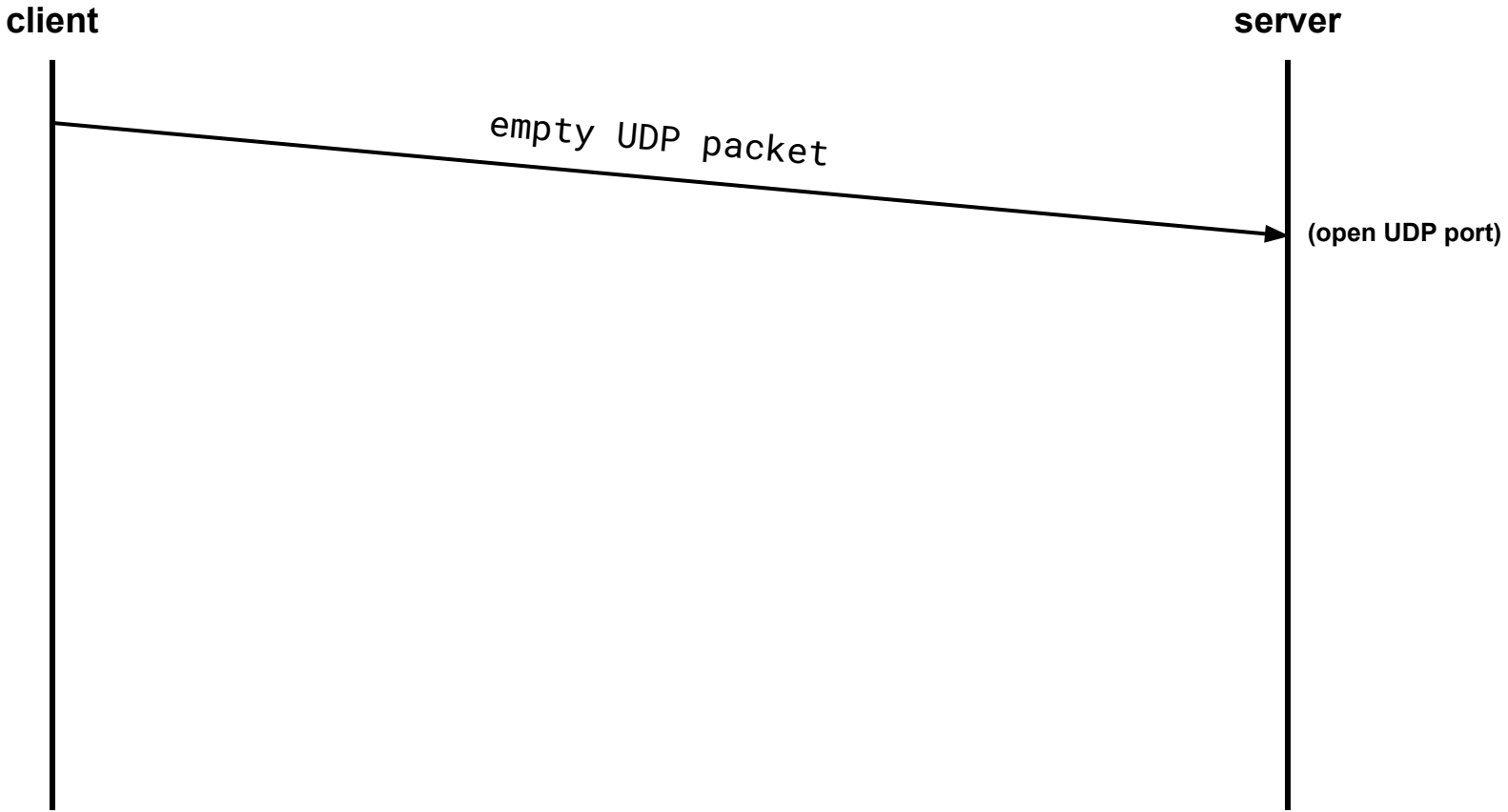
2 Active Reconnaissance - TCP (open port)



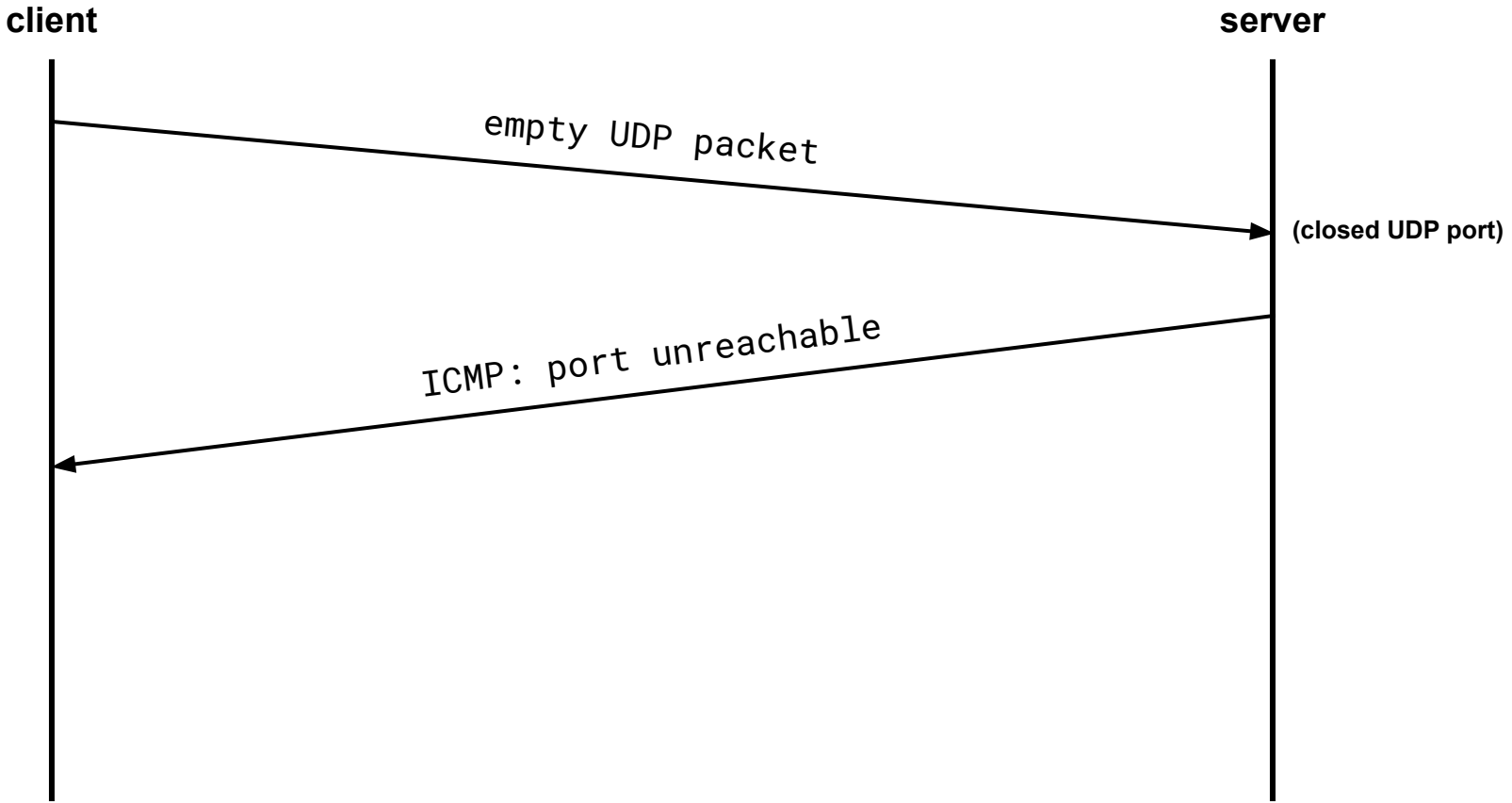
2 Active Reconnaissance - TCP (closed port)



2 Active Reconnaissance - UDP (open port)



2 Active Reconnaissance - UDP (closed port)



2 Active Reconnaissance - Port Range Specification

Switch	Method
default behavior	top 1000 ports
`-F`	top 100 ports
`--top-ports <n>`	top `n` ports
`-p80,400-420`	port `80`, `400`, `401`, …, `420`
`-p-`	all 65535 ports

2 Active Reconnaissance - Port 0/tcp

- note, that the lowest possible value for a port is (theoretically) 0
  - a normal socket API will not allow port zero to be used
  - instead, it will interpret port 0 as a wild card (random ports)
- although invalid, nothing stop you from specifying it in the header field
  - trojans sometimes will listen on port 0 to not appear on most port scans
- in Nmap, to scan it, will have to explicitly specified:

```
-p0-65535
```

2 Active Reconnaissance - Port States

- from the view of a Nmap scan, there is a total of 6 possible states a port can be in
  - `open`
  - `closed`
  - `filtered`
  - `unfiltered`
  - `open|filtered`
  - `closed|filtered`

2 Active Reconnaissance - Port Scan Types

- there is a variety of scan modes for port scanning, with the three most well-known (and useful) ones being
  - TCP Connect Scan: `-sT` [default for unprivileged]
  - TCP SYN Scan: `-sS` [default for privileged]
  - UDP Scan: `-sU`

2 Nmap - Port Scanning Flags

```
# banner grabbing
[user@kali ~]$ nmap <ip> -sV

# OS detection
[user@kali ~]$ nmap <ip> -O

# output and formats
[user@kali ~]$ nmap <ip> -oN <filename>
[user@kali ~]$ nmap <ip> -oG <filename>
[user@kali ~]$ nmap <ip> -oX <filename>
[user@kali ~]$ nmap <ip> -oA <filename>
```

2 Nmap - NSE - Introduction

- while the standard `nmap` command offer many useful functionalities, there is a mechanism by which one extend them even further
- the Nmap Script Engine (NSE) allows for incorporating custom scripts and functionalities into Nmap
  - written in Lua

2 Nmap - NSE - Location

- all the NSE scripts that are shipped with nmap can be found in `/usr/share/nmap/scripts`
  - the `script.db` file inside of this directory works as a catalog

```
[user@kali ~]$ head /usr/share/nmap/scripts/script.db
Entry { filename = "acarsd-info.nse", categories = { "discovery", "safe", } }
Entry { filename = "address-info.nse", categories = { "default", "safe", } }
Entry { filename = "afp-brute.nse", categories = { "brute", "intrusive", } }
Entry { filename = "afp-ls.nse", categories = { "discovery", "safe", } }
Entry { filename = "afp-path-vuln.nse", categories = { "exploit", "intrusive", "vuln", } }
Entry { filename = "afp-serverinfo.nse", categories = { "default", "discovery", "safe", } }
Entry { filename = "afp-showmount.nse", categories = { "discovery", "safe", } }
Entry { filename = "ajp-auth.nse", categories = { "auth", "default", "safe", } }
Entry { filename = "ajp-brute.nse", categories = { "brute", "intrusive", } }
Entry { filename = "ajp-headers.nse", categories = { "discovery", "safe", } }
```

2 Nmap - NSE - Script Categories

- each script is at least part of two categories

categorization regarding its intrusiveness

Category	Description
`safe`	script is not expected to crash any service, is not expected to overwhelm services with too many requests and does not exploits any vulnerabilities
`intrusive`	opposite of `safe`

2 Nmap - NSE - Script Categories

- each script is at least part of two categories

categorization regarding its functionality

Category	Description
`auth`	authentication and access control
`brute`	execution of brute-forcing attacks
`discovery`	information gathering
`exploit`	execution of exploits
`version`	collection of version information
`vuln`	detection of vulnerabilities

2 Nmap - NSE - `--script`

- to use NSE, need to supply the `--script` flag

```
[user@kali ~]$ nmap --script script1 target.com
[user@kali ~]$ nmap --script script1,script2 target.com
[user@kali ~]$ nmap --script all target.com
[user@kali ~]$ nmap --script discovery target.com
[user@kali ~]$ nmap --script http-* target.com

# use --script-help instead of --script to get instructions
# of a specific script (how, what, which arguments)
[user@kali ~]$ nmap --script-help script1
```

2 Nmap - NSE - `default` Category

- the `default` category contains fundamental scripts, that can be executed by simply supplying the `-sC` switch
- for a script to be part of the `default` category, it will need to have following properties

Property	Explanation
Quick	script should not take too long execution
Useful	script should generally provide useful insights
Not too Verbose	script should not produce too much output
Reliable	script should work consistently
Non-Intrusive	script should not be intrusive
Private	script should not propagate information to a 3rd-party

2 Nmap - `-A` Switch

- the `-A` switch combines the functionalities of the following three switches:
  - `-sV` → banner grabbing
  - `-O` → OS detection
  - `-sC` → use script of category `default`

2 Nmap - NSE - Recommendation

- here is my recommendation on how to scan a specific target

1. identify open TCP ports

```
[user@kali ~]$ sudo nmap -p- -vvv -Pn -sS -oN tcp_all <target>
```

2. enumerate open TCP ports

```
[user@kali ~]$ sudo nmap -pX,Y,... -vvv -Pn -sSCV -oN tcp_enum <target>
```

3. identify open UDP ports (and enumerate after)

```
[user@kali ~]$ sudo nmap -F -vvv -Pn -sU -oN udp <target>
```

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3 Active Reconnaissance - Service Enumeration

- goals of enumeration:
  - which services run on open ports
  - software and version
  - finding usernames
  - finding passwords
  - identifying attack surface
  - preparation for attack

3 Active Reconnaissance - Service Enumeration

- information we are trying to gather
  - banner grabbing
  - credentials
  - information used later
    - what services are running
    - configurations
- best case: we identify how to perform exploitation to gain a initial foothold on a system

3 Active Reconnaissance - Service Enumeration

- in this section, we will go over some of the most common services that we can encounter and how to enumerate them
  - by default, certain services have reserved ports on which they run, but they can be hosted on any port
    - check banner to find out which service
    - tools meant for enumeration or interaction of certain protocol will use their default port, but we can always explicitly define port if necessary
  - if you discover port, but cannot identify service, try connecting manually to it using netcat or telnet

```
[user@kali ~]$ nc <target> <port>
[user@kali ~]$ telnet <target> <port>
```

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.1 Service Enumeration - FTP - Characterization

- FTP = File Transfer Protocol
- server uses `21/tcp` for command transfer and `20/tcp` for data transfer
- used for transfer of file between client and server
- cleartext communication
- usually required username and password for authentication
- outdated protocol
- common FTP server implementations
  - vsFTPd
  - ProFTPD
  - Pure-FTPd
  - FileZilla

3.1 Service Enumeration - FTP Server Context

- the FTP server runs in the context of some user
  - often, some specific account will be created for that user, e.g., the `ftp` user
- accounts for the FTP server are, by default, separate from system accounts, but…
- FTP servers can be configured to allow for system users to log in with their system credentials

3.1 Service Enumeration - FTP - `ftp`

- `ftp` is a CLI utility that acts as a FTP client and allows us to easily connect to and use a FTP server
  - otherwise we would need to use the FTP communication protocol, which is a bit more tedious

```
[user@kali ~]$ ftp [<username>@]<target> [-p <port>]
Connected to <target>.
220 Welcome to vsFTPd 3.0.5
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

3.1 Service Enumeration - FTP - `ftp` Commands

```
# basics
ftp> ls
ftp> cd
ftp> less

# downloading a file
ftp> get <remote-file>
# uploading a file
ftp> put <local-file>

# close FTP connection
ftp> bye

# recursively download FTP folder content
[user@kali ~]$ wget -m ftp://<user>:<pass>@10.0.0.3/
```

3.1 Service Enumeration - FTP - Banner Grabbing

- to start off, we will try to gather which software and version is used to run the FTP server

```
[user@kali ~]$ ftp <target>
Connected to <target>.
220 Welcome to vsFTPd 3.0.5
Name (<target>:user):
```

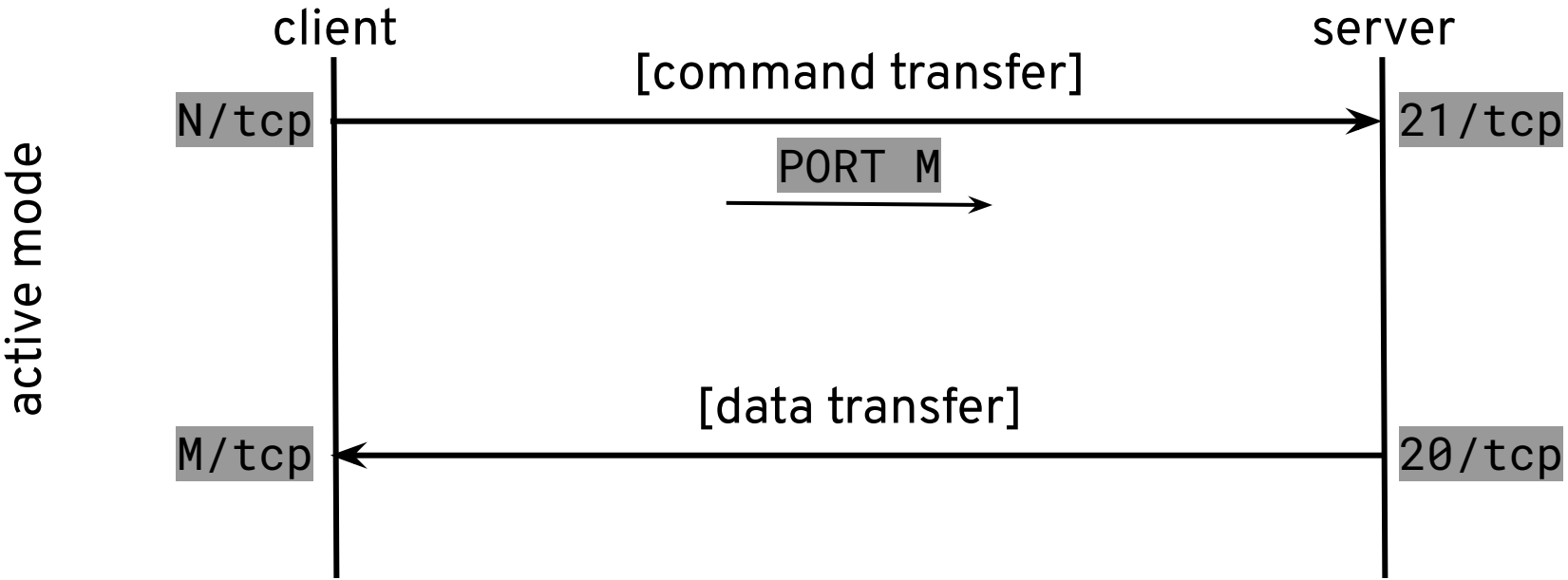
```
[user@kali ~]$ nc <target> 21
220 (vsFTPd 3.0.5)
```

3.1 Service Enumeration - FTP - Active vs. Passive Move

- FTP has two modes it can run in: active and passive mode
  - these mode differentiate in how the data transfer channel is established

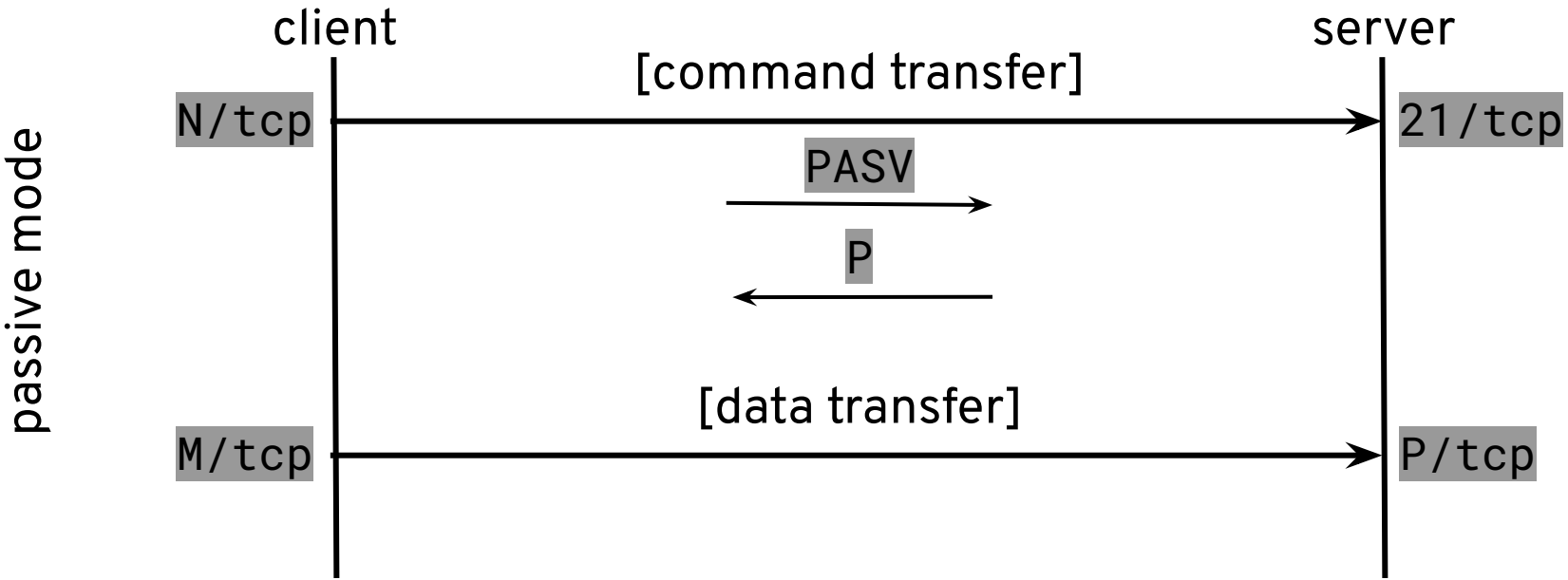
3.1 Service Enumeration - FTP - Active vs. Passive Move

- FTP has two modes it can run in: active and passive mode
  - these mode differentiate in how the data transfer channel is established



3.1 Service Enumeration - FTP - Active vs. Passive Move

- FTP has two modes it can run in: active and passive mode
  - these mode differentiate in how the data transfer channel is established



3.1 Service Enumeration - FTP - Active vs. Passive Move

```
# enable passive mode
ftp> passive on

# enable active mode
ftp> passive off
```

3.1 Service Enumeration - FTP - Anonymous Authentication

- FTP servers have possibility of allowing anonymous authentication, i.e., something similar to guest accounts
- the default credentials to make use of this feature are `anonymous:` (anything)

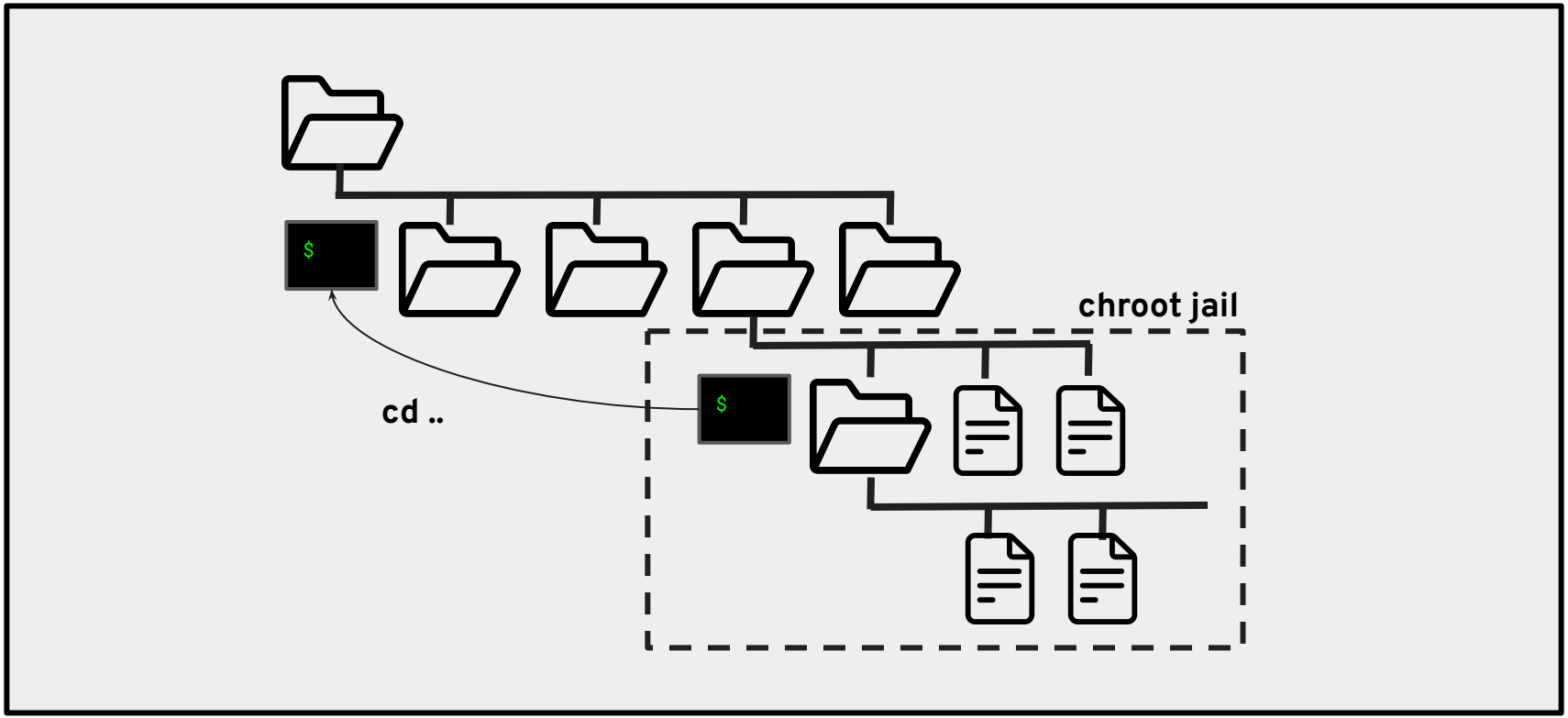
```
[user@kali ~]$ ftp anonymous@<target>
Connected to <target>.
220 Welcome to vsFTPd 3.0.5
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

3.1 Service Enumeration - FTP - Directory Traversal

- **directory traversal** attack (also known as **path traversal**) aims to access files and directories that are stored outside the intended folder
  - can also be found in other services, e.g., HTTP
- FTP servers use a mechanism called **chroot jail**, which confines users to a specific directory
  - if not correctly implemented, may be able to escape this restricted environment using path traversal techniques
- to actually verify this, check if you can move out of the ftp directory

```
ftp> cd ..
```

3.1 Service Enumeration - FTP - Directory Traversal



3.1 Service Enumeration - FTP - Config File

- the configuration of the FTP server is stored in a file
  - e.g., for vsFTPd it would usually be at /etc/vsftpd.conf
- some configuration settings can be dangerous (as we have seen)

```
anonymous_enable=YES
anon_upload_enable=YES
anon_mkdir_write_enable=YES
anon_root=/home/username/ftp
chown_uploads=YES
chown_username=username
local_enable=YES
no_anon_password=YES
write_enable=YES
chroot_local_user=NO
```

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.2 Service Enumeration - SSH - Characterization

- SSH = Secure Shell
- allows for encrypted interaction with remote system via a shell
- by default, service available on port `22/tcp`
- requires system credentials to do so
- not much to get from, except its software / version
  - but gives us a foothold on the system if we discover system credentials
- most common software implementation used is OpenSSH
  - also known as OpenBSD Secure Shell and is developed as part of the OpenBSD operating system

3.2 Service Enumeration - SSH - Banner Grabbing

```
[user@kali ~]$ ssh [<username>@]<target> [-p <port>]
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.5
<username>@<target> password:
```

```
[user@kali ~]$ nc target-recon 22
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.5
```

3.2 Service Enumeration - SSH - Authentication

- there are two common ways to handle authentication in SSH

1. password authentication
   - user logs in with username and password
   - simple, but less secure (especially with weak password)
2. public key authentication
   - client generates a key pair (public and private key)
   - public key is placed on the server, and the private key remains with the user
   - more secure than passwords, since you need private key
   - private key can even be password secured

3.2 Service Enumeration - SSH - Private Keys

- the private SSH key of a `<user>` is usually stored at: `/home/<user>/.ssh/<private-key-name>`
- the name of the private key can, theoretically be set to anything, but most of the time, the default name is kept, which depends on the type of asymmetric key algorithm used (extended lists):
  - RSA → `id_rsa`
  - DSA → `id_dsa`
  - ECDSA → `id_ecdsa`
  - ED25519 → `id_ed25519`
- if we get our hands on a private SSH key → might be our initial foothold

3.2 Service Enumeration - SSH - Private Keys

```
[user@kali ~/.ssh]$ cat id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABl
FQ/9weMCV2B5hfOG/wSRX4Klk7Ft3poHwc7gdX8IoG0YnerLtJt7mUOrR
V3nrWHTsV2mglN2oVApuuLSqpGZRde/rRzwm8dzAiqZJpMqgA3q7ufz9b
72eiAeOn4AeVmy2q7s2zO/JTjMCQf0+EL30+rTzN4b7A7h7+kV87mJ51v
...
AlyyvIcbEOfRrglniMgg0qV4+iLAAAAMEA42E6bez7I7AzrwVu7n+Hhm5
uAFCExOkW4ZtJ7nXetRQu5lXUcxkVzKtqFMZCsTY5QhXUrcEB8bxh3LMW
HsV8fKX4Afm0asLfQYllzSqKV3VWznSlIZbK60ygONePL2F6aeeXetR3d
5fEICmfhQ6srnrAi6wqQUUuMAAAARYXR0YWNrZXJAYXR0YWNrZXIBAg==
-----END OPENSSH PRIVATE KEY-----
```

3.2 Service Enumeration - SSH - Public Keys

- like private SSH key, the public key of a `<user>` is usually stored at: `/home/<user>/.ssh/<public-key-name>`
- public keys follow the same naming convention as the private key, with the difference being the file extension `.pub`
- server side, the file `/home/<user>/.ssh/authorized_keys` contains a list of public keys that are authorized to log in as that user

3.2 Service Enumeration - SSH - Public Keys

```
[user@kali ~/.ssh]$ cat id_rsa.pub
---ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQC62AcehEHjSDZMlr47haVnS3Ij2
0qjxbfCsZTawvC72H7zPqxnliZKivJkSPf5l3yHnib/gOWaZXS3ORKc1b
Y/7HzekbOf1lCpxTcYCcjDSyQakaeLrbCg1LLLKLwIGCUEyOzYm2UDPWQ
dyHFvuLrji8g+nC8m/xy11uh62udrdE0Brvhn3njB9lYcnArHJK5bcgLN
5MeRkXbXxC4G/imzP4dtGaXJ6OeciZE5S+eeuD1k0t9I/9w/VU6QUdx0u
QQ1cbJtNjVJwrhbCOBhEQ+BCl63PE55CUIpW7MD9m0KvCFQOkwxeDVfmf
tvP+Uz8QDl6QnLFlyf511Y6q039Xn3xHok0tz/MSzTUjkYpn2XyUyYEHF
l/5H5fT+XhkiJYlO8kmOKz019dWLzdqnrlYK8HrmYaeZz+BhRWuHCNXXL
a7LmyBYKE0PZNDuZJhjZY/bY6CD17/OwU0Co5PCm81KD9FxQbcJ/6n7/e
Xj2r4OLDzSbdu6aTHGCDFyHPl49CcE= user@kali
```

3.2 Service Enumeration - SSH - Key Generating

- to generate a key pair, use the `ssh-keygen` utility
- you can define
  - location of keys
  - whether you want to password protect your private key
  - which key algorithm to use
  - bit size of the generate keys (more → secure)

```
[user@kali ~]$ ssh-keygen [-t (rsa | … | ed25519)] -b [bits]
```

3.2 Service Enumeration - SSH - `known_hosts`

- can be seen as your personal certificate authority
- list of all SSH server host public keys you have trusted
- each entry consists of
  - server names, IP address
  - type of public key the server uses
  - public key data
  - (and optionally, comment data)
- if public key data of remote server changes, might be a man-in-the-middle attack
  - you will be notified
- location: `/home/<user>/.ssh/known_hosts`

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.3 Service Enumeration - DNS - Characterization

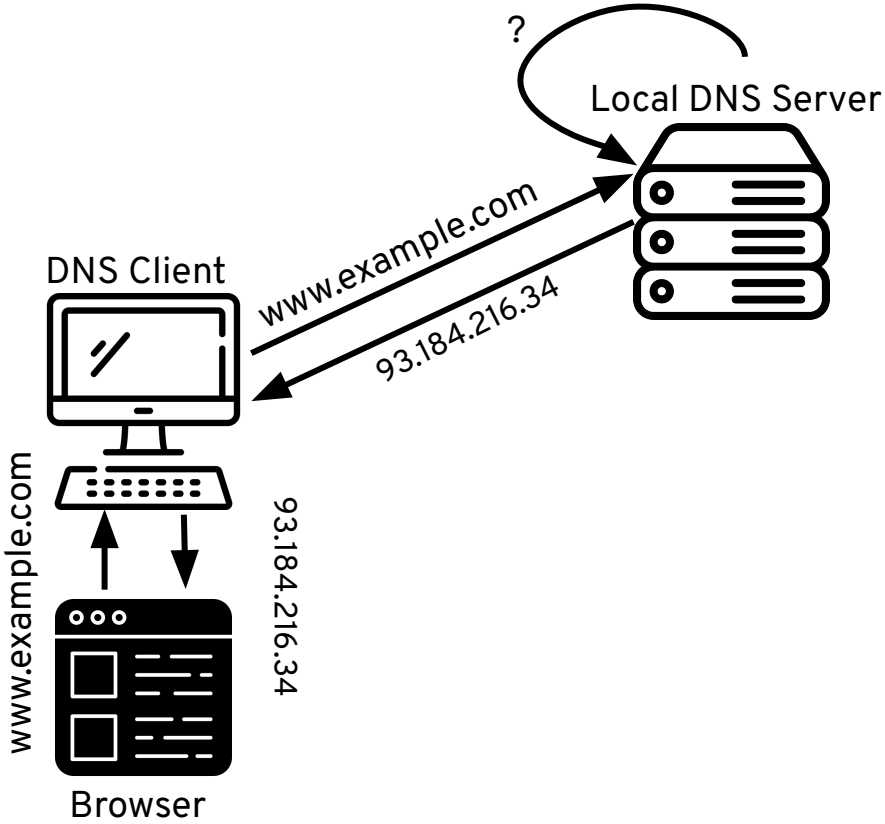
- DNS = Domain Name System
- associates multiple pieces of information with a domain name
  - most notably: IP addresses
- by default, service available on port `53/tcp` and `53/udp`
  - `53/udp` used for DNS queries
  - `53/tcp` used for zone transfers
- by default, no authentication and no encryption
- can provide sensitive information about the infrastructure
  - systems that are associated with a domain name

3.3 Service Enumeration - DNS - Resolution Process (Linux)

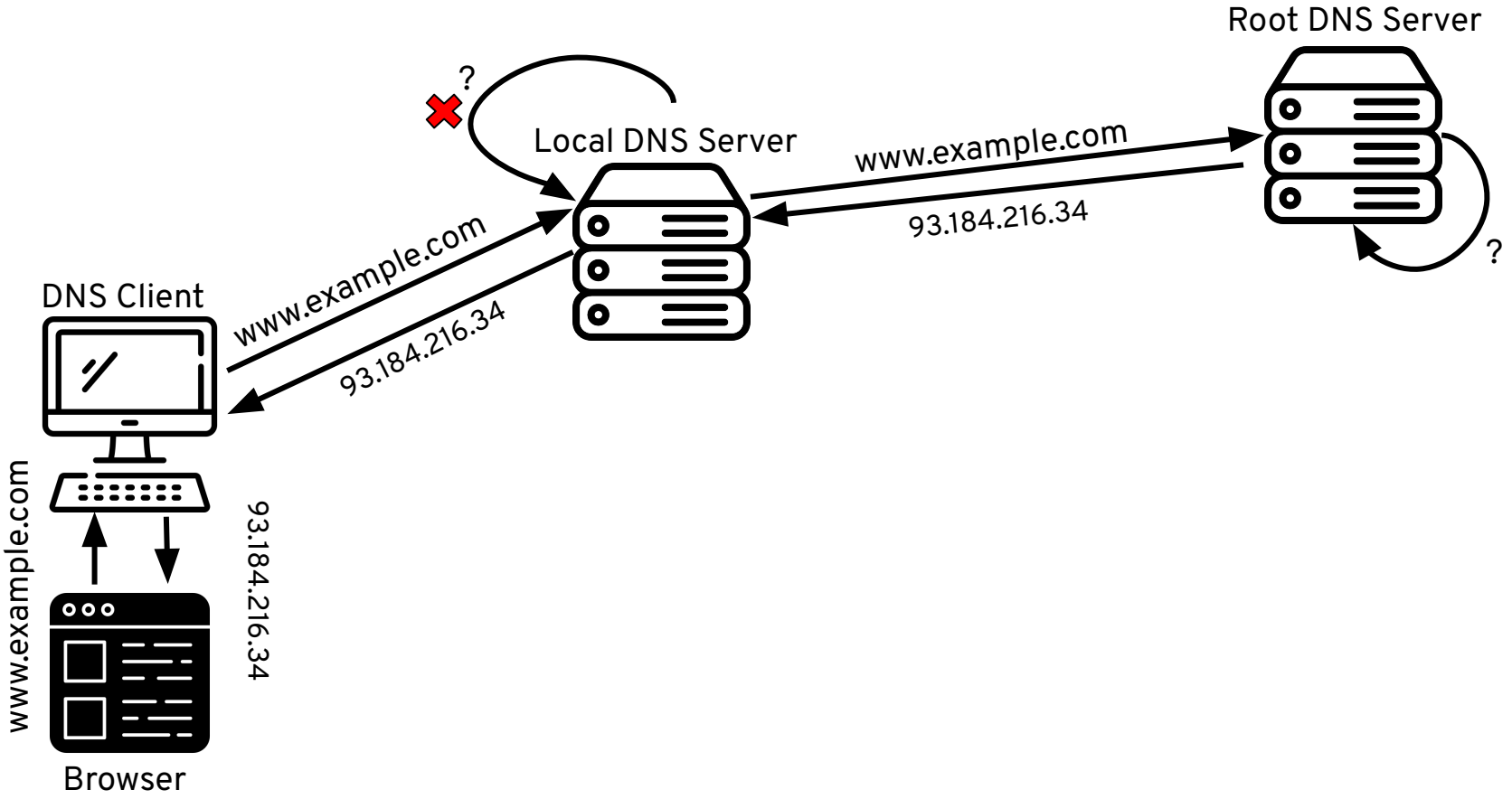
- you enter `www.example.com` into your browser; what happens?

1. browser checks it cache for IP address
2. (local) DNS resolver client is contacted
   a. it checks whether the entry can be found in `/etc/hosts`
   b. if not, it will then query its local DNS server

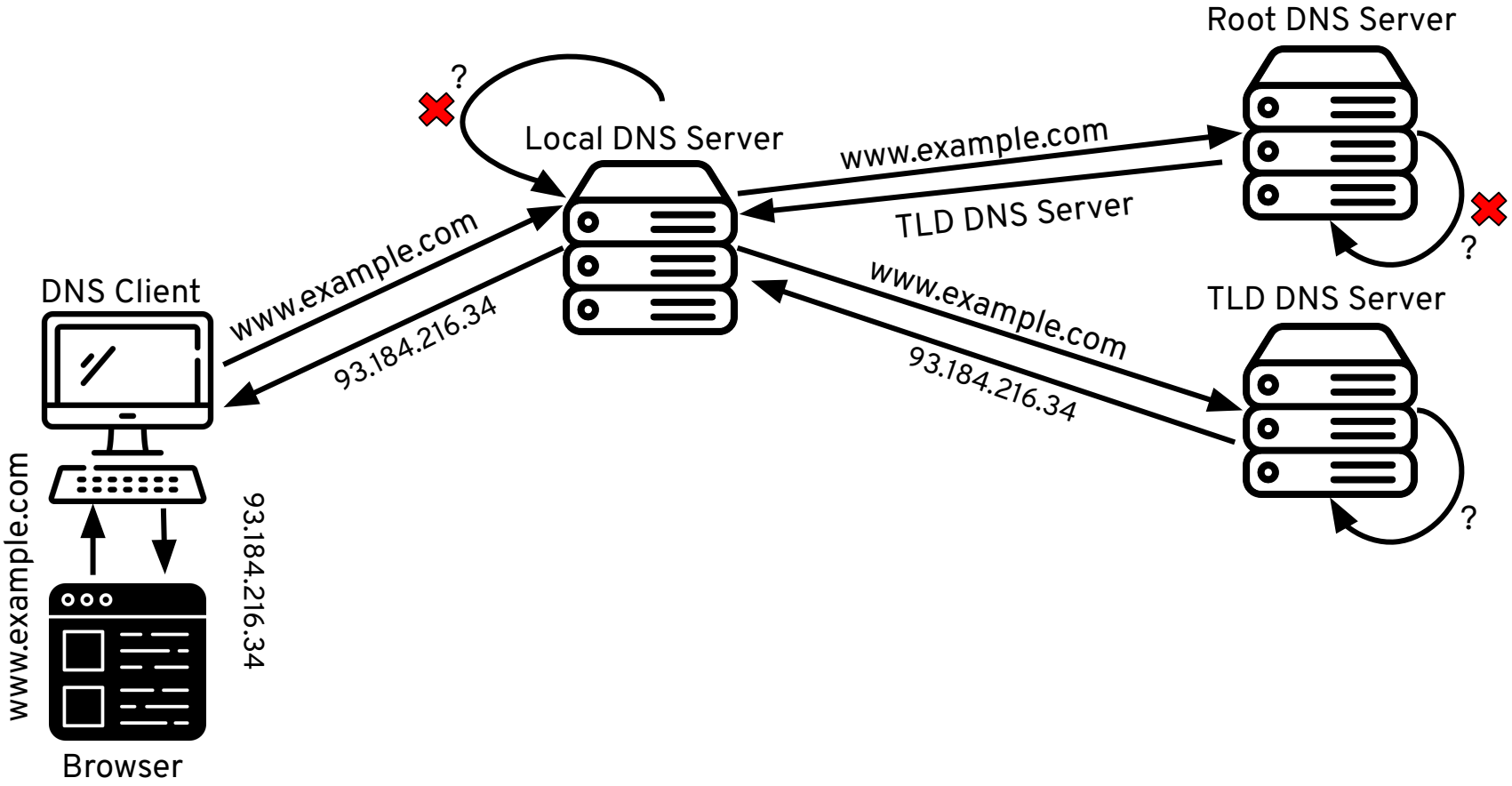
3.3 Service Enumeration - DNS - Resolution Process (iterative)



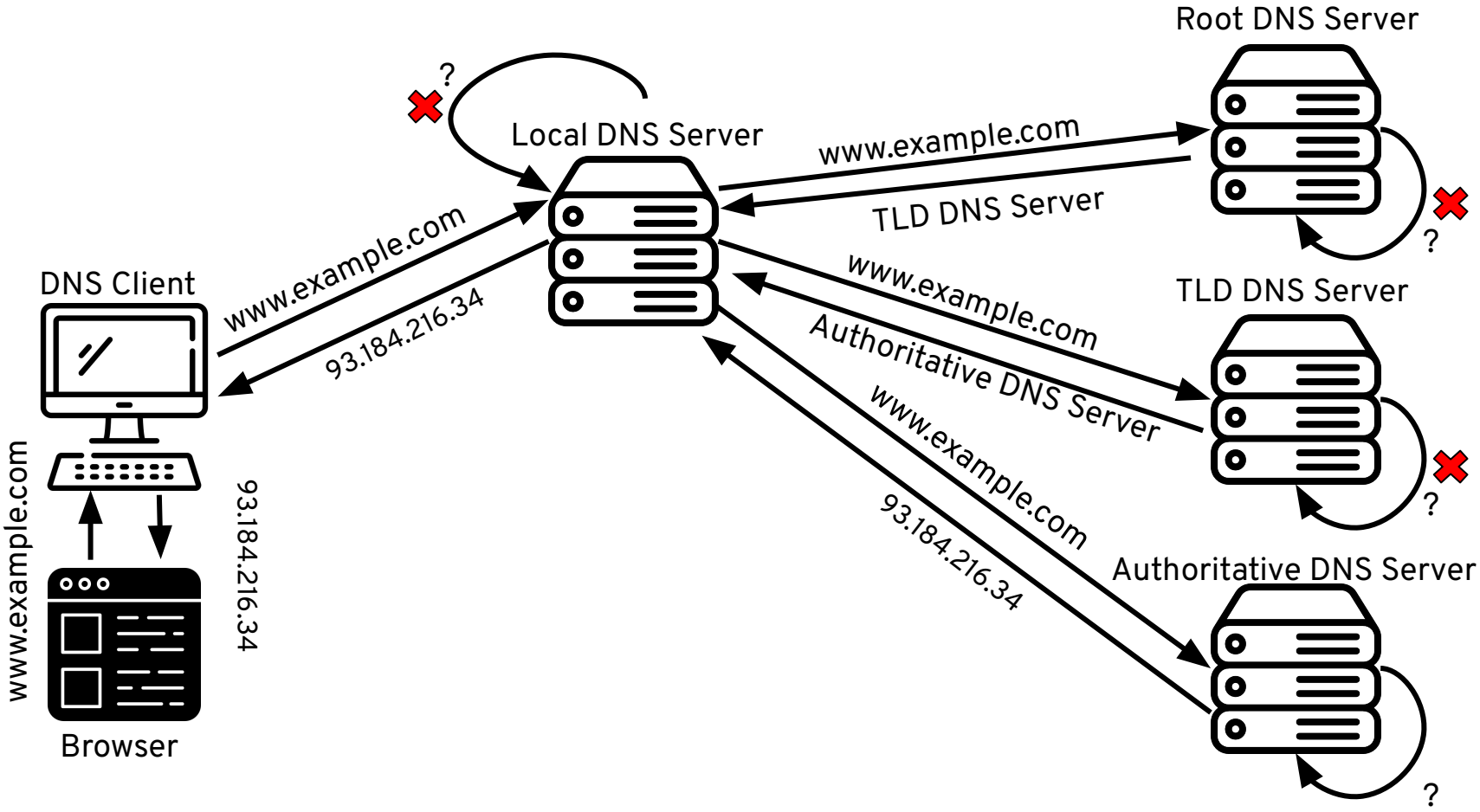
3.3 Service Enumeration - DNS - Resolution Process (iterative)



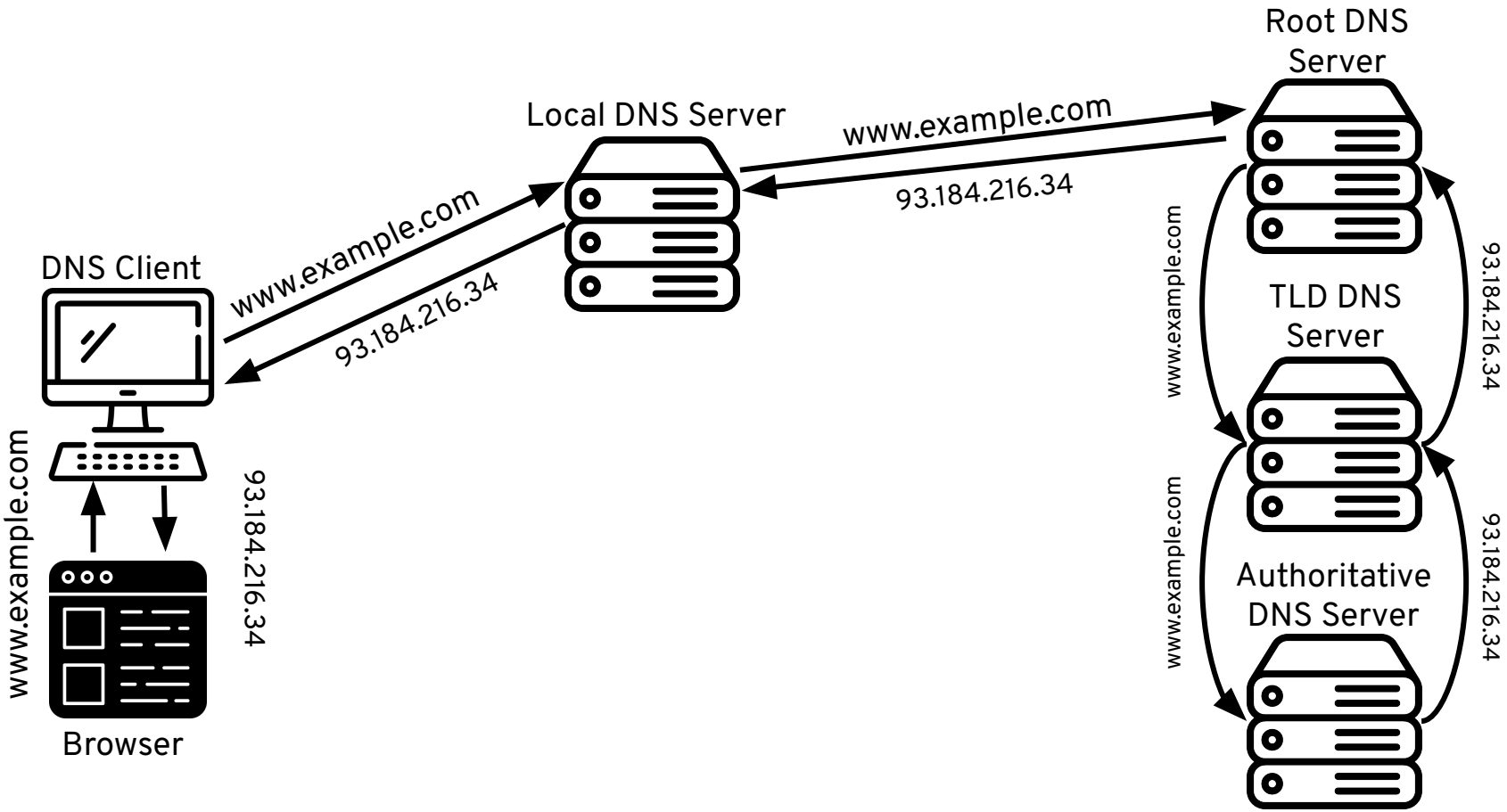
3.3 Service Enumeration - DNS - Resolution Process (iterative)



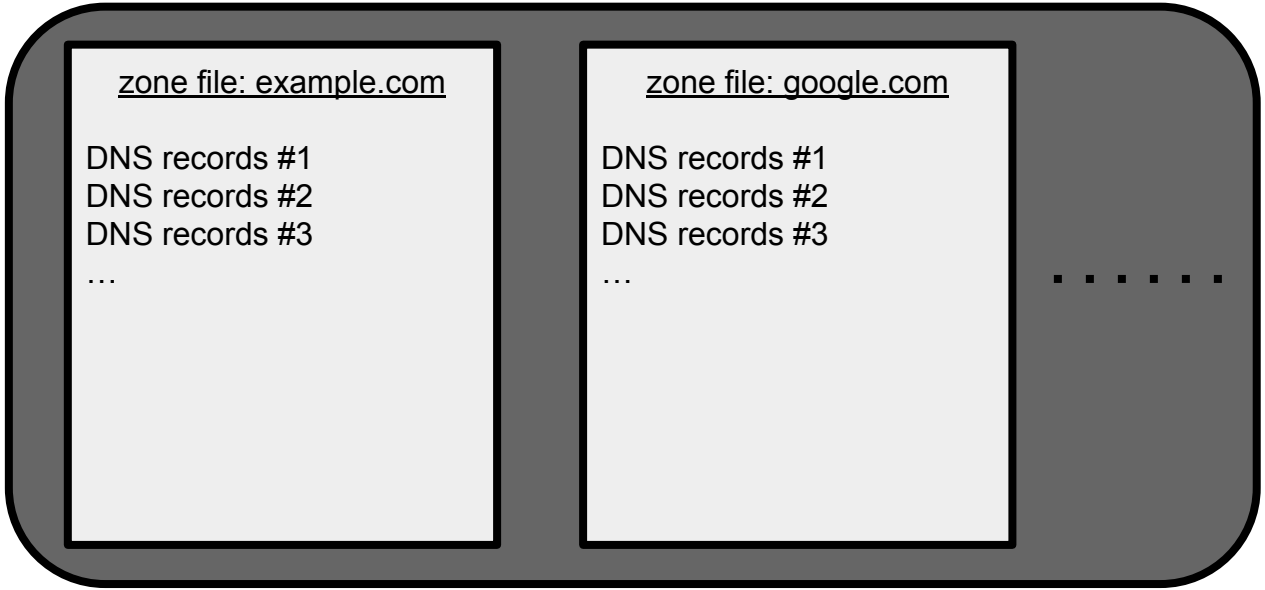
3.3 Service Enumeration - DNS - Resolution Process (iterative)



3.3 Service Enumeration - DNS - Resolution Process (recursive)



3.3 Service Enumeration - DNS - Servers & Records



DNS Server

DNS Record Blueprint:

Name	TTL	Type	Data

3.3 Service Enumeration - DNS - Record Types

Types of Records	Description
`A`	IPv4 address of a domain
`AAAA`	IPv6 address of a domain
`CNAME`	alias for another domain name
`MX`	mail server of a domain
`NS`	name server of a domain
`PTR`	domain name for reverse lookup
`SOA`	administrative information about domain
`TXT`	text notes

3.3 Service Enumeration - DNS - CLI for DNS Queries

- in order to issue DNS requests from the command line interface, you can use the `dig` and `host` utilities
- both of these fulfill similar functions, which is why we will only focus on one of them: `dig`

3.3 Service Enumeration - HTTP/HTTPS - `dig` Basic Usage

```
# general command syntax
[user@kali ~]$ dig [@server] [domain] [type] [options]

# record queries
[user@kali ~]$ dig <domain>
[user@kali ~]$ dig <domain> AAAA
[user@kali ~]$ dig <domain> CNAME
[user@kali ~]$ dig <domain> MX
[user@kali ~]$ dig <domain> NS
[user@kali ~]$ dig <domain> SOA
[user@kali ~]$ dig <domain> TXT

# PTR record query, i.e., reverse DNS lookup
[user@kali ~]$ dig -x <ip>

# all records
[user@kali ~]$ dig <domain> ANY
```

3.3 Service Enumeration - DNS - Zone Transfer

- a DNS Zone Transfer is a mechanism, allowing a primary DNS server to copy a zone file to a secondary DNS server
  - maintain redundancy
  - load balancing
- two types
  - AXFR (full zone transfer): secondary DNS server requests entire zone file from primary
  - IXFR (incremental zone transfer): secondary DNS server requests only records that have changed since last transfer

3.3 Service Enumeration - DNS - Zone Transfer Security

- since a zone file contains all DNS records for a specific domain name, it is sensitive information that should be protected
  - can even contain private entries
- can restrict zone transfer by, e.g., configuring primary DNS server to allow zone transfer only to specific IP addresses
- to try and request a DNS zone transfer manually, we can use the syntax below

```
# DNS zone transfer request
[user@kali ~]$ dig axfr @<server> <domain>
```

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.4 Service Enumeration - HTTP/HTTPS - Source Code

- the source code of a website can give valuable information
  - comments
  - JavaScript source code
- basically all browser provide mechanisms to investigate
  - display source code
  - web developer tools (`CTRL SHIFT I` in Firefox)

3.4 Service Enumeration - HTTP/HTTPS - Directory Fuzzing

- web server publishes a certain part of the file system
- generally, all the files and directories can be accessed via the URL
  - server might be configured to hide/secure certain elements
- no way of listing all the files and directories that are available
- only way: guess file names and see what the web server returns
  - known as **directory fuzzing**
- goal is to find resources that should not be available or endpoints that can help us in attacking the target
  - login pages
  - web frameworks
  - sensitive data
  - ...

3.4 Service Enumeration - HTTP/HTTPS - Directory Fuzzing

- variety of tools for performing directory fuzzing
  - `dirb` (C)
  - `dirbuster` (Java)
  - `gobuster` (Go)
  - `wfuzz` (Python)
  - `feroxbuster` (Rust)
  - `ffuf` (Go)
- different properties (speed, user-friendliness, features, …)
- personally, I am using `ffuf`
  - the tool I am going to showcase to you
  - pre-installed on Kali Linux

3.4 Service Enumeration - HTTP/HTTPS - ffuf Basic Usage

```
# perform basic directory fuzzing
[user@kali ~]$ ffuf -u http://<target>/FUZZ -w <word-list>

# examples with variation
[user@kali ~]$ ffuf -u https://<target>/FUZZ -w <word-list>
[user@kali ~]$ ffuf -u http://<target>/tmp/FUZZ -w <word-list>
[user@kali ~]$ ffuf -u http://<target>/FUZZ/readme.txt -w <word-list>

# output with color highlighting (easier readability)
[user@kali ~]$ ffuf -u http://<target>/FUZZ -w <word-list> -c

# perform recursive directory fuzzing, i.e., perform directory
# fuzzing in identified directories
[user@kali ~]$ ffuf -u http://<target>/FUZZ -w <word-list> -recursion

# verbose output, printing full URL and redirect location (if any)
with the results.
[user@kali ~]$ ffuf -u http://<target>/FUZZ -w <word-list> -v
```

3.4 Service Enumeration - HTTP/HTTPS - ffuf Example Output

```
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
------------------------------------------------

 :: Method           : GET
 :: URL              : http://target/FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
------------------------------------------------


.hta                    [Status: 403, Size: 277, Words: 20, Lines: 10, Duration: 0ms]
.htpasswd               [Status: 403, Size: 277, Words: 20, Lines: 10, Duration: 0ms]
.htaccess               [Status: 403, Size: 277, Words: 20, Lines: 10, Duration: 0ms]
index.html              [Status: 200, Size: 10717, Words: 3498, Lines: 365, Duration: 1ms]
robots.txt              [Status: 200, Size: 82, Words: 4, Lines: 7, Duration: 1ms]
server-status           [Status: 403, Size: 277, Words: 20, Lines: 10, Duration: 0ms]
```

3.4 Service Enumeration - HTTP/HTTPS - `ffuf` Match & Filtering

- even if requested file does not exist → we get an answer
  - typically a `404 Not Found`
  - we do not want to see this
- `ffuf` lets us define what we want to see (matches) and what we do not want to see (filters)
  - default: match response status code below:

    `200-299,301,302,307,401,403,405,500`

	HTTP code	# lines	regex	size	# words
match	`-mc X`	`-ml X`	`-mr X`	`-ms X`	`-mw X`
filter	`-fc X`	`-fl X`	`-fr X`	`-fs X`	`-fw X`

- choice of wordlist is crucial
- should consider questions likes the ones below when choosing:
  - which technology does the server use?
  - which language is your target website written in?
  - which industry is this website dedicated to?
  - does the website use a specific naming scheme?
- even if it is essential to choose the right list for the job, having some "default" wordlists you will start with, is no problem
- my personal choice:
  - `common.txt` (~5k entries)
  - `DirBuster-2007_directory-list-2.3-medium.txt` (~220k entries)

3.4 Service Enumeration - HTTP/HTTPS - Seclists

- "Seclists" is huge a collection of wordlists
  - directory fuzzing
  - usernames
  - passwords
  - ...
- the `seclists` package can be installed to place this collection in the path `/usr/share/wordlists/`
  - can be pre-installed on Kali Linux, but doesnt have to be
- directory fuzzing wordlists located in `/usr/share/wordlists/seclists/Discovery/Web-Content/`

3.4 Service Enumeration - HTTP/HTTPS - HTTP Headers

- while we will focus on this in the topic of web exploitation, another point to check are HTTP headers
- standard HTTP response headers can give information about the used technology
- furthermore, it is possible to configure custom headers, which may contain valuable information
  - custom headers usually start with `x-` or `X-`

```
# make a http HEAD request
[user@kali ~]$ curl -I http://<target>/
HTTP/2 200
server: nginx
...
x-my-header: value
```

3.4 Service Enumeration - HTTP/HTTPS - `robots.txt`

- the World Wide Web is constantly being crawled by different entities for different reasons
  - e.g., by Google for their search engine
- `/robots.txt` → file to communicate with the crawler
  - which endpoints should not be crawled
  - where to find the sitemap
    - overview over all pages on a website (useful for Google)
    - typically named: `sitemap.xml`

3.4 Service Enumeration - HTTP/HTTPS - `robots.txt` Example

```
User-agent: Googlebot
Disallow: /nogooglebot/

User-agent: *
Allow: /

Sitemap: https://www.example.com/sitemap.xml
```

3.4 Service Enumeration - HTTP/HTTPS - vhosts

- originally (`HTTP/1.0`), a single web server was only able to host a single website per IP and port
- why?
  - → imagine you are hosting two websites via a web server and you receive the following HTTP request

```
GET /index.html HTTP/1.0
```

- which of the two websites is supposed to be referenced here?
  - issue: original HTTP requests did not contain the whole URL (with the domain name) but only the requested path

3.4 Service Enumeration - HTTP/HTTPS - vhosts

- solution: vhost = virtual host
- introduced with `HTTP/1.1`
- requires that each HTTP requests contains a `host` header that contains the domain that it is meant to reach

```
GET /index.html HTTP/1.0
Host: example.com
```

3.4 Service Enumeration - HTTP/HTTPS - vhosts

- might encounter situations, where trying to reach a web server via its IP (`http://127.0.0.1/`) does not yield proper results
  - you might require the correct `host` header to be set

- <u>how to find out the correct host header</u>
  - circumstantial evidence (e.g., you found a company name somewhere on a different port)
  - TLS certificates can also help; their CN or SAN fields list the domain names expected in the Host header, since the certificate must specify which domains it's valid for

3.4 Service Enumeration - HTTP/HTTPS - vhosts

- might encounter situations, where trying to reach a web server via its IP (`http://127.0.0.1/`) does not yield proper results
  - you might require the correct `host` header to be set

- <u>how to set the `host` header</u>
  - if you try to simply replace the IP by the domain name, you might end up with a domain name that is not resolvable
  - instead, you can add an entry to the `/etc/hosts` file

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.5 Service Enumeration - SMB - Characterization

- SMB = Server Message Block
- protocol used for file sharing
- by default, service available on port `445/tcp`
- standard service on Windows systems
  - enabled by default
- can also operated on Linux
  - Samba
- Versions
  - SMB v1.x (very vulnerable, e.g., EternalBlue)
  - SMB v2.x
  - SMB v3.x

3.5 Service Enumeration - SMB - Share

- SMB share, or file share, is a shared resource made accessible
- usually, a share represents a directory
  - but generally, can be any type of resource, like a printer
- properties (for directory share):
  - share name
  - path (local filesystem path to directory that will be shared)
  - permissions: whether the share is writable and by whom
  - guest access: whether guest users can access the share
- shares can be mounted
- special share: `IPC$` (used for network communication and control, rather than file sharing)

3.5 Service Enumeration - SMB - NBT

- NBT = NetBIOS over TCP/IP
- typically hosted on `139/tcp`
- allows NetBIOS services (like name resolution and file sharing) to work over TCP/IP
- SMBv1 is depending on NBT in order to perform file sharing operations over the network
- enabled by default on Windows systems
- usually always appears together with `445/tcp`

3.5 Service Enumeration - SMB - NetBIOS Records

- NBT can store information as NetBIOS Records
- interesting for us is that we can get hostname and domain/workgroup

Code	Name
00	Workstation Service
03	Windows Messenger Service
06	Remote Access Service
20	File Service (Host Record)
21	Remote Access Service Client
1B	Domain Master Browser
1D	Browser Election

3.5 Service Enumeration - SMB - NBT Enumeration

```
[user@kali ~]$ nmblookup -A <target>
   hostname      <00> -          B <ACTIVE>
   hostname      <03> -          B <ACTIVE>
   hostname      <20> -          B <ACTIVE>
   ...
```

```
[user@kali ~]$ nbtscan -r <target-ip>
IP address       NetBIOS Name     Server    User             MAC address
------------------------------------------------------------------------------
<taget-ip>       hostname         <server>  hostname         00:00:00:00:00:00
```

3.5 Service Enumeration - SMB - smbclient - Connect

- solid manual tool for enumerating SMB shares is `smbclient`

listing shares with `smbclient`

```
[user@kali ~]$ smbclient -N -L <target>
[user@kali ~]$ smbclient -U [domain/]username[%password] -L <target>

    Sharename       Type    Comment
    ---------       ----    -------
    myshare         Disk
    IPC$            IPC     IPC Service (Samba Server)
```

3.5 Service Enumeration - SMB - smbclient - Connect

- solid manual tool for enumerating SMB shares is `smbclient`

interactive shell with `smbclient`

```
[user@kali ~]$ smbclient -N //<target>/<share>
[user@kali ~]$ smbclient -U [domain/]username[%password] //<target>/<share>

Try "help" to get a list of possible commands.
smb: \>
```

3.5 Service Enumeration - SMB - smbclient - Connect

- solid manual tool for enumerating SMB shares is `smbclient`

commands in interactive shell with `smbclient`

```
smb: \> ls                    # list files
smb: \> cd                    # change directory
smb: \> get <remote-file>     # download a file to local directory
smb: \> put <local-file>      # upload a file to remote SMB share

smb: \> recurse ON            # recursively download all files
smb: \> prompt OFF
smb: \> mget *
```

3.5 Service Enumeration - SMB - enum4linux

- `enum4linux` is a tool that aggregates much of the enumeration techniques that were shown here and more
  - hostname & workgroup/domain
  - NBT information
  - NULL session / guest access check
  - OS information
  - user enumeration through RID cycling
  - share enumeration
  - password policy
- many functionalities, but the base command is a good start

```
[user@kali ~]$ enum4linux -a <target>
```

Reconnaissance

"BECAUSE EVEN HACKERS NEED A GAME PLAN"

1	Introduction to Reconnaissance	
2	Active Recon - Scanning	
3	Active Recon - Enumeration	
	3.1	FTP
	3.2	SSH
	3.3	DNS
	3.4	HTTP/HTTPS
	3.5	SMB
	3.6	SNMP

3.6 Service Enumeration - SNMP - Characterization

- SNMP = Simple Network Management Protocol
- used to monitor/manage network devices (routers, switches,...)
  - SNMP agent → runs on a managed devices and collects data and processes information of this device
  - SNMP manager → runs central systems and collects data from SNMP agents
- by default, service available on port `161/udp` (and `162/udp` for SNMP traps)
- by default, no authentication and no encryption

3.6 Service Enumeration - SNMP - Versions

- there are multiple SNMP versions:
  - SNMPv1:
    - authentication is based on a string (community string)
    - communication in plain-text
  - SNMPv2 & SNMPv2c:
    - similar to SNMPv1
  - SNMPv3
    - better authentication (e.g., requires username and password)
    - encrypted communication

3.6 Service Enumeration - SNMP - Community Strings

- two types of community strings
  - public → basically read only
  - private → read and write
- server will not respond to false community string
  - if responds → valid community string
- default community string: `public`
- can be brute-forced with hydra using Seclists' `common-snmp-community-strings.txt`

```
[user@kali ~]$ hydra -P \
/usr/share/wordlists/seclists/Discovery/SNMP/common-snmp-community-strings.txt \
<target> snmp
```

3.6 Service Enumeration - SNMP - MIB

- all the parameters and variables that can be monitored/controlled on a device are stored in a hierarchical structure call the Management Information Base (MIB)
  - independent format, so there is a unified way for storing device information across manufacturers
  - basically a text file which can be queried
  - does not contain data, but instructs where to find it
- each element in this structure is referred to as an object
  - can represents CPU usage, network interface status, ...

3.6 Service Enumeration - SNMP - OID

- each object in the MIB is identified by a unique OID (Object Identifier)
- sequence of numbers separated by dots, e.g., 1.3.6.1.2.1.1.1
  - numbers represent a specific path in the MIB tree to locate the particular information of a resource
- some of these resources are standard, but others can be defined by private companies to manage their resources

3.6 Service Enumeration - SNMP - Tools

- there is a variety of tools for performing SNMP enumeration
- here, we will showcase two of these
  - `snmp-check`
    - generic SNMP tool that dumps raw OID data
    - good for deep, manual enumeration when you know which OIDs to inspect
    - verbose and low-level
  - `snmpwalk`
    - queries common/interesting OIDs and parses them into human-readable facts (host, users, interfaces, routes, processes)
    - faster for reconnaissance.

```
[user@kali ~]$ snmp-check <target> -c <community-string>
...
```

```
[user@kali ~]$ snmpwalk -v <snmp-version> -c
<community-string> <target> .
iso.3.6.1.2.1.1.1.0 = STRING: "Linux system
6.8.0-41-generic #41-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug
2 20:41:06 UTC 2024 x86_64"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.8072.3.2.10
iso.3.6.1.2.1.1.3.0 = Timeticks: (2501) 0:00:25.01
...
```

(Re)Sources

- https://nmap.org/book/toc.html
- https://github.com/ffuf/ffuf
- https://book.hacktricks.wiki/en/index.html